# NAAN MUDHALAVAN

## PROJECT REPORT

# AI FAQ ASSISTANT API

An Intelligent FAQ and Customer Support Automation System Using Artificial Intelligence and REST API

Prepared for Academic Project Submission

Academic Year 2026–2027

TEAM DETAILS


| S.NO | TEAM | NAME | EMAIL ID |
|---|---|---|---|
| 1 | Team Leader | Oviya M | Oviya9385@gmail.com |
| 2 | Team Member | Kaviya C | Kaviyamurugammal13@gmail.com |
| 3 | Team Member | Kaviya V | Vkaviya666@gmail.com |
| 4 | Team Member | Kavya P | Kaviyapushparaj7@gmail.com |
| 5 | Team Member | Leenadevi S | leenadevisugumaran@gmail.com |

# TABLE OF CONTENTS


1. Project Overview
2. Scenario-Based Case Study
3. Software Requirements
4. Hardware Requirements
5. Project Architecture
6. Database / Entity-Relationship Description
7. Key Features
8. Roles and Responsibilities
9. User Flow and MVC Architecture
10. Phase 1 – Brainstorming & Ideation
11. Phase 2 – Requirement Analysis
12. Phase 3 – Project Design
13. Phase 4 – Project Planning
14. Phase 5 – Project Development
15. Phase 6 – Project Testing
16. Phase 7 – Project Documentation
17. Phase 8 – Project Demonstration
18. API Testing Specification

# 1. PROJECT OVERVIEW

AI FAQ ASSISTANT AGENT is an AI-powered FAQ and customer support automation backend project. It is designed as a secure REST API platform for FAQ creation, user authentication, AI-powered content management, semantic search, and automated question-answer generation.

The system is built using Node.js and Express.js. MongoDB is used as the primary database with Mongoose ODM for data modelling and schema validation. JWT is used for stateless authentication and route protection, while bcrypt is used for password hashing. Google Gemini AI is integrated for AI-assisted FAQ generation and question-answering.

# 2. SCENARIO-BASED CASE STUDY

## Background

Priya is a customer support operations manager who handles a large number of repetitive user inquiries. She faces difficulties in organizing entries, classifying topics, preparing precise summaries, and tracking client question trends efficiently.

## Problem Statement

- Difficulty generating and maintaining up-to-date documentation workflows.
- Limited visibility into semantic search match keywords and rapid query resolutions.
- Time-consuming manual processes for drafting complex support answers.
- Lack of unified database permission layers protecting private documentation from public adjustments.

## Proposed Solution

The AI FAQ ASSISTANT AGENT provides a centralized and secure API layer that separates application business logic from frontend delivery systems. It supports content creation, category management, database persistence, authentication, semantic search, and automated AI text generation.

## Expected Outcome

The system reduces operational delays, maintains data structural integrity through schema validation, protects modification operations using security tokens, and improves FAQ discoverability.

## 3. SOFTWARE REQUIREMENTS

- Operating System: Windows 10/11, macOS, or Linux.
- Node.js: Version 18 or above.
- npm: Version 9 or above.
- Express.js: Version 4/5 ecosystem.
- MongoDB with Mongoose ODM.
- Postman or ThunderClient for API testing.
- Visual Studio Code or similar code editor.

## 4. HARDWARE REQUIREMENTS

- Processor: Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or equivalent.
- RAM: Minimum 8 GB; 16 GB recommended for concurrent MongoDB processing.
- Storage: Minimum 1 GB available workspace.

## 5. PROJECT ARCHITECTURE

### Interface

API tools such as Postman query distinct route paths and receive JSON entities.

### Express Server Gateway

Monitors network ports, maps GET, POST, PUT and DELETE methods, parses JSON payloads, and handles CORS headers.

### Authentication Middleware

Extracts authorization tokens, validates token signatures, and checks authorization for protected routes.

### Granular Services

Enforces isolated domain rules for user sessions and operational query modifications.

### Database Interface Layer

Mongoose translates application logic into MongoDB queries.

### AI Flow

Google Gemini SDK integration supports question-answering and automated topic/FAQ generation.

## 6. DATABASE / ENTITY-RELATIONSHIP DESCRIPTION

### User Entity

- _id – Primary Key (ObjectId)
- name – String, Required
- email – String, Required, Unique
- password – Hashed, Required
- role – String, Default: user
- createdAt
- updatedAt

### FAQ Entity

- _id – Primary Key (ObjectId)
- question – String, Required
- answer – String, Required
- category – String, Required
- createdBy – Reference to User Collection
- createdAt
- updatedAt

### AI Generation Entity (Logical)

- topic
- generatedQuestion
- generatedAnswer
- generatedCategory
- generatedAt

### Category Entity (Logical)

- categoryName
- description
- totalFAQs

### Key Structural Mappings

- User → FAQ: One-to-Many. A registered user can create and manage multiple FAQ records.
- Category → FAQ: One-to-Many. Each category can contain multiple FAQ documents.
- User → AI Generated FAQ: One-to-Many. A user can generate multiple AI-assisted FAQs.
- FAQ → Search Results: Logical relationship used to retrieve matching FAQ documents.

## 7. KEY FEATURES

- JWT Authentication and Authorization.
- AI FAQ Generation using Google Gemini AI.
- Semantic / keyword-based FAQ Search.
- Complete CRUD operations for FAQ records.
- Category Management.
- Password Encryption using bcrypt.
- Centralized Error Handling.
- Mongoose Schema Validation.
- RESTful API Architecture.
- MongoDB Integration using Mongoose ODM.

## 8. ROLES AND RESPONSIBILITIES

### Admin

- Manage registered users.
- View, update and delete FAQs.
- Monitor AI-generated content.
- Manage FAQ categories.
- Access complete system functionality.

### Content Creator

- Create FAQs.
- Edit personal FAQ records.
- Delete owned FAQs.
- Generate FAQs using AI.
- Organize FAQs under categories.

### Authenticated User

- View public FAQs.
- Search FAQs using keywords.
- Generate AI-assisted answers.
- Access personal profile information.

### Public User

- View published FAQs.
- Search FAQs.
- Cannot create, edit or delete FAQ records.
- No access to protected API endpoints.

## 9. USER FLOW AND MVC ARCHITECTURE

The system follows a layered backend architecture and uses the MVC pattern to separate data definitions, execution logic, and API routing.

### Model Layer

Defines Mongoose schemas, field rules, defaults, and validation before database persistence.

### Controller Layer

Processes requests, validates input, delegates operations to models/services, and returns JSON responses.

### View / Routing Interface Layer

This headless backend does not use traditional UI views; API routes act as the interface layer.

## 10. PHASE 1 – BRAINSTORMING & IDEATION

**Project Title:** AI FAQ ASSISTANT AGENT

**Idea:** Develop an AI-assisted FAQ and customer support automation backend.

**Problem Identified:** Repetitive support questions, manual answer preparation, difficulty organizing FAQs, and limited search visibility.

**Proposed Idea:** Create a secure REST API that supports FAQ management, search, authentication, and AI-generated FAQ content.

**Primary Technology:** Node.js, Express.js, MongoDB, Mongoose, JWT, bcrypt, Google Gemini AI.

## 11. PHASE 2 – REQUIREMENT ANALYSIS

**Functional Requirements:** User registration/login, authentication, FAQ CRUD, category organization, FAQ search, AI FAQ generation, protected routes, and error handling.

**Non-Functional Requirements:** Security, data validation, reliability, scalability, maintainability, and efficient data retrieval.

**Software Requirements:** Node.js 18+, npm 9+, Express.js, MongoDB/Mongoose, Postman or ThunderClient, VS Code.

**Hardware Requirements:** Intel Core i5 8th Gen+/Ryzen 5 equivalent, 8 GB RAM minimum, 1 GB workspace.

## 12. PHASE 3 – PROJECT DESIGN

**Architecture:** Layered REST API architecture with Express gateway, authentication middleware, service/controller logic, Mongoose database layer, and Gemini AI integration.

**Database Design:** User, FAQ, AI Generation (logical), and Category (logical) entities.

**Security Design:** JWT route protection and bcrypt password hashing.

**API Design:** RESTful endpoints using GET, POST, PUT and DELETE operations.

## 13. PHASE 4 – PROJECT PLANNING

**Development Plan:** Set up the project, install dependencies, create server/app structure, configure database, implement authentication, implement FAQ APIs, integrate AI generation, test endpoints, document the system, and prepare demonstration.

**Source Structure:** Root project folder contains a src folder with server, app, middleware, models, routes and related backend modules as described in the project document.

**Testing Plan:** Use Postman or ThunderClient to verify public and protected endpoints and validate request/response behaviour.

## 14. PHASE 5 – PROJECT DEVELOPMENT

**Project Setup:** Create the project folder and src subfolder. Initialize npm and install Express, Mongoose, dotenv, CORS, JWT, bcrypt and Google GenAI packages.

**Backend Modules:** Develop server gateway, Express setup, authentication middleware, centralized error middleware, User and FAQ schemas, authentication routes, FAQ routes, and AI routes.

**AI Integration:** Connect the controller/service flow with Google Gemini SDK for automated FAQ/question-answer generation.

## 15. PHASE 6 – PROJECT TESTING

**Testing Tool:** Postman or ThunderClient.

**Registration Test:** POST /api/auth/register

**Login Test:** POST /api/auth/login

**FAQ Creation Test:** POST /api/faqs with Bearer JWT token.

**Search Test:** GET /api/faqs/search?q=configure

**AI Generation Test:** POST /api/ai/generate-faq with Bearer JWT token.

**Expected Result:** Valid requests should return appropriate JSON responses, while protected routes should require valid authentication.

## 16. PHASE 7 – PROJECT DOCUMENTATION

**Documentation Contents:** Project overview, case study, requirements, architecture, database/entity description, features, roles, user flow, MVC architecture, setup, development, API endpoints, testing, and demonstration.

**Submission Repository:** Create a Public GitHub repository and upload project files phase-wise according to the eight SmartBridge submission phases.

**Team Details:** Team members: _____

## 17. PHASE 8 – PROJECT DEMONSTRATION

**Demo Format:** Record a complete project demonstration video and upload it to Google Drive with access set to Anyone with the link can view.

**Demo Must Include:** Project name, purpose, uses/benefits, screen sharing, voice-over explanation, project execution/working process, and final output.

**Demo Project Name:** AI FAQ ASSISTANT AGENT

## 18. API TESTING SPECIFICATION

API endpoints can be tested using Postman or ThunderClient.

User Registration

POST /api/auth/register

Example request:

```
{
  "name": "Priya Sharma",
  "email": "priya@example.com",
  "password": "securepassword123"
}
```

User Login

POST /api/auth/login

Example request:

```
{
  "email": "priya@example.com",
  "password": "securepassword123"
}
```

Create FAQ

POST /api/faqs

Example request:

```
{
  "question": "What is AI FAQ Agent?",
  "answer": "AI FAQ Agent is a system that helps users create, manage, search and generate frequently asked questions using AI.",
  "category": "General"
}
```

FAQ Search

GET /api/faqs/search?q=AI

AI FAQ Generation

POST /api/ai/generate-faq

Example request:

```
{
  "topic": "Artificial Intelligence"
}
```

## REFERENCE

AI FAQ ASSISTANT AGENT provides a centralized backend solution for secure FAQ management and AI-assisted customer support. By combining RESTful APIs, MongoDB/Mongoose, JWT security, bcrypt password hashing, search functionality, and Google Gemini AI integration, the project supports structured FAQ creation, retrieval, management, and automated content generation.